The Chinese Wall Security Model in Security Enhanced Linux

Experimentation-Based with Significant Software Development

Calum Cardownie, Ross Horne, Marc Roper

Aim

We aim to implement a Dynamic Chinese Wall security policy in the SELinux environment

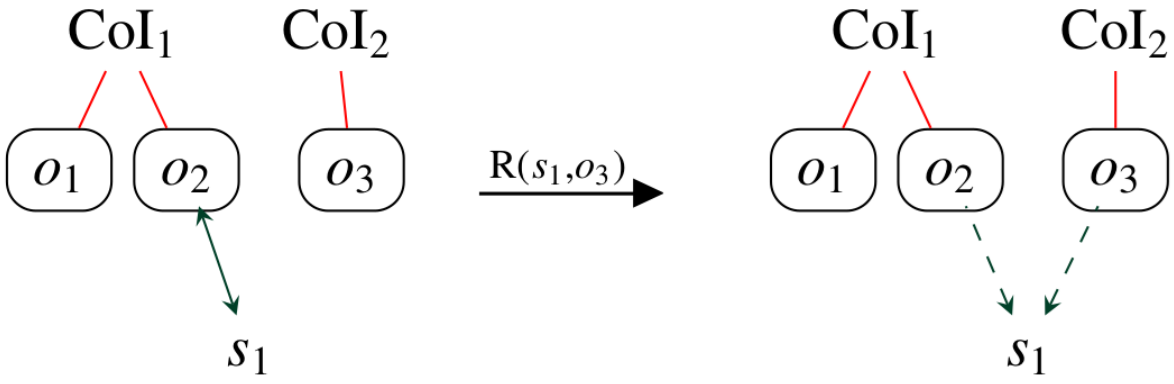
Objectives

- Create and setup a Red Hat virtual machine
- Create a static implementation of a Chinese Wall
- Develop a Daemon capable of detecting Conflict of Interest breaches and subjects access history
- Achieve a dynamic Chinese Wall by enforcing SELinux policies upon CoI breach detection and

Summary of Related Work

- Chinese Wall Policy

- Chinese Wall / Brewer Nash model
  - Aggressive Chinese Wall model



- Security Enhanced Linux
  - Theoretical Chinese Wall in UNIX

Summary of Related Work

- Chinese Wall in adjacent Computing systems
  - Cloud Computing, on hypervisor level
  - Workflow Management Systems, on application level

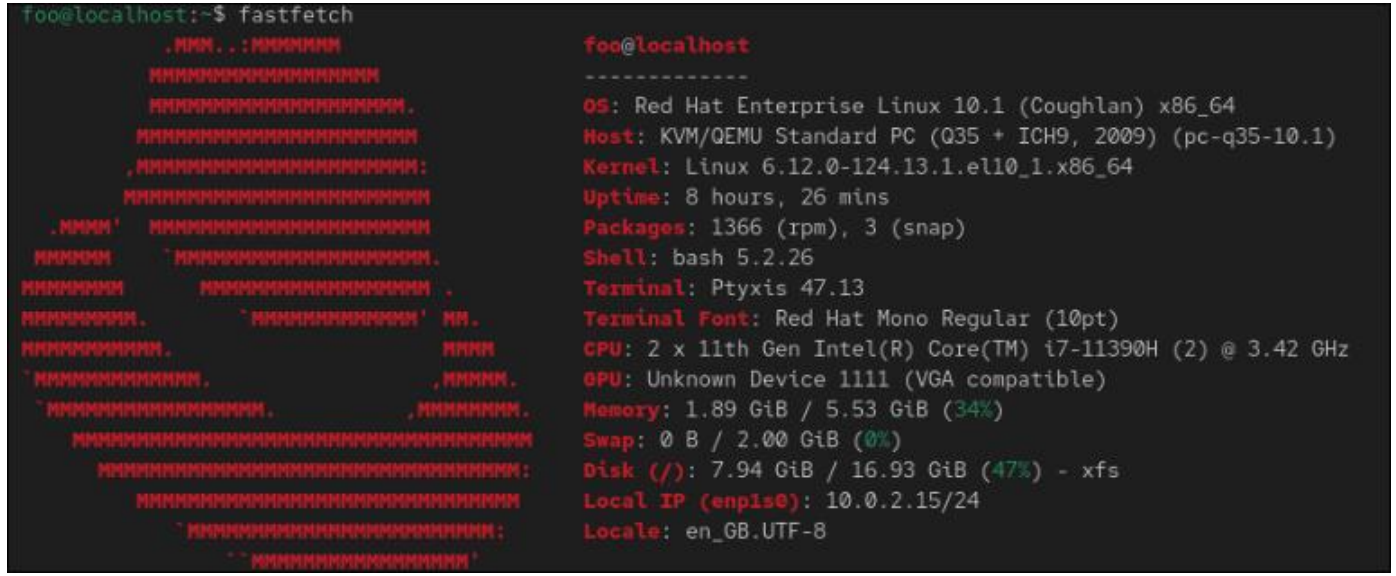
Project Specification

- Chinese Wall Daemon must
  - Observe all subjects in the system and their access to objects
  - Identify CoI breaches in the system, and track history of subjects reads & writes
  - Attempt policy enforcement, leveraging SELinux's AVC and Security Server to achieve a Chinese Wall

- Written in Golang
  - Memory safety
  - Gouroutines, Channels

Progress to date

- Red Hat virtual machine setup



- Static Implementation of Chinese Wall created

Static Chinese Wall in SELinux

Made by creating foo_daemon, bar_daemon and using SELinux policies to restrict bar_daemon from accessing foo's secret files. These are not restricted with discretionary controls

```
// Bar's daemon

#include <unistd.h>
#include <stdio.h>

FILE *f;

int main(void){
  while(1){
    f = fopen("/home/testgrounds/static/secret","r");
    sleep(5);
    fclose(f);
  }
}
```

```
policy_module(bar_daemon, 1.0.0)

########################################
#
# Declarations
#

type bar_daemon_t;
type bar_daemon_exec_t;
type secret_t;
init_daemon_domain(bar_daemon_t, bar_daemon_exec_t)

permissive bar_daemon_t;

########################################
#
# bar_daemon local policy
#
allow bar_daemon_t self:fifo_file rw_fifo_file_perms;
allow bar_daemon_t self:unix_stream_socket create_stream_socket_perms;
neverallow bar_daemon_t secret_t:file { read open getattr };

domain_use_interactive_fds(bar_daemon_t)

files_read_etc_files(bar_daemon_t)

miscfiles_read_localization(bar_daemon_t)
```

```
/home/testgrounds/static ❯ ls -Z | grep secret
unconfined_u:object_r:secret_t:s0 secret
```

```
/home/testgrounds/static ❯ ps -efZ | grep bar_daemon
system_u:system_r:bar_daemon_t:s0 root     74313       1  0 20:36 ?        00:00:00 /usr/local/bin/bar_daemon
```

```
avc:  denied  { read } for  pid=74313 comm="bar_daemon" name="secret"
```

Challenges to date

- Several attempts made to restrict based on user
  - Users, roles, categories were all changed
    - SELinux main restrictions are on the type, not the former
  - The process created from a user still had its own type independent of the user
    - Custom shell to spawn processes with specific domains
  - Using the category to restrict users did not work
    - Also did not use SELinux policies

Approach to Development

- Methodology
  - Agile development throughout the project, as needs and end goal may be adjusted based on feasibility of completion
- Design
  - A compiled binary for the daemon
  - Fairly simple structure
- Implementation and Technology ideas
  - Golang, eBPF for kernel, LSM for hooks to read and write operations

Testing and Evaluation plans

- Stress test the daemon, multiple subjects running accessing things inside and outside of CoI groups, complete with a log of the daemon's actions

- Evaluated as a success if a Chinese Wall can be created efficiently

Thank you for listening